

CSCE 713 SOFTWARE SECURITY · MODULE 13 · LECTURE NOTES

Claiming Code Security Honestly

What an assessment of code establishes.

MODULE OUTCOMES

- State what each technique establishes.
- Identify the standard overclaims.
- Write a checkable assessment claim.
- Place this course relative to CSCE 701 and CSCE 711.
- State a proportionate programme for a real codebase.

1 What each technique establishes

Technique	What it establishes	What it does not establish
Static analysis	That these rules, run on this code, produced these findings.	Absence of defects — it is unsound by design (Module 07 §1), so silence carries very little information.
Fuzzing	That this coverage was achieved, over this duration, with no crashes under this oracle.	Anything at all about unreachable code (Module 08 §3) — which is why coverage is the number that matters.
Sanitisers	That these executions, on these inputs, contained no detected violation.	Anything about any other input (Module 09 §4).
Code review	That a person looked at this code for these specific things.	That they would have recognised the defect if it were there (Module 11 §4).
A test suite	That these behaviours are as the tests expect.	Anything adversarial — the tests encode the expected cases, which are precisely not the interesting ones.

Not one of these establishes absence, and the middle column is in every case a statement about *what you did* rather than about the program — which is the whole content of this module and the reason the right-hand column is worth memorising. **The asymmetry is fundamental rather than a limitation of current tools** (CSCE 627, and Module 07 §1).

“THE SCANNER FOUND NOTHING” IS A STATEMENT ABOUT THE SCANNER

It says that these particular rules, applied to this code, produced no findings — which is information about the rules' coverage at least as much as about the code's quality. A scanner with no rules finds nothing on any input.

And it is routinely reported as 'the code is clean', which is a different claim entirely and is not supported by the evidence — and the substitution happens silently, usually between the engineer and the summary slide.

The honest form names the method and its scope: 'this ruleset, on these files, found no instances of these classes' — all four clauses checkable by somebody else, which is the test of a real claim.

Which is exactly the same correction CSCE 701 Module 13 §1 makes about a penetration test, and CSCE 711 Module 13 §3 makes about a security proof — three courses in one semester identifying the same error: substituting the absence of a finding for the presence of a property.

2 Overclaims, and defensible claims

The claim	The correction
'The code is secure.'	Not a supportable statement. Against which adversary, for which defect classes, in which components?
'We fuzzed it.'	With what coverage, for how long, with which oracle, and from what corpus? (Module 08 §3.)
'Static analysis is clean.'	Which rules, and what is the known false negative profile of those rules? (Module 07 §1.)
'It passed security review.'	Who looked, at what, for what, and for how long? (Module 11 §4's recording requirement.)
'It is memory safe.'	In safe code — name the unsafe blocks and the foreign function boundary (Module 02 §4).
'We have 100% test coverage.'	Line coverage says nothing whatsoever about adversarial inputs. See the note.

The coverage row is the most seductive of these — a complete-sounding number that measures which lines were executed by friendly inputs, and says nothing at all about hostile ones. A function with 100% line coverage can have an unhandled integer overflow on every path, because the tests supplied reasonable values. This is the overclaim engineers believe most readily, because it is quantitative and it is their own number.

- 'We fuzzed the parser for 400 CPU-hours under AddressSanitizer and UBSan, reaching 78% of its edges, with no findings in the final 300 hours.' Every clause checkable, and the plateau is itself informative.

- **'These twelve static analysis rules run on every change; the eight findings to date are triaged in the repository with reasons.'** (Module 07 §3.)
- **'The authorisation layer was reviewed by two engineers against this written checklist; the three findings are fixed and the checklist is in the repository.'** (Module 11 §4.)
- **'Memory safe except for these four unsafe blocks and the FFI boundary, each of which was reviewed individually.'** — the narrow claim of Module 02 §4, made precisely.
- **And the limitation statement: 'we have no coverage of concurrency defects, and no coverage of authorisation logic outside this layer.'** The limitation statement is the one thing that distinguishes an assessment from a tool run — and it is **Project 2's hardest-graded requirement**, because it requires knowing what your methods structurally cannot see.

3 The semester

Course	What it covers	The honest summary
CSCE 701	Systems, operations, architecture, and people.	Where the incidents are — credentials, misconfiguration, missing authorisation, unpatched dependencies, phishing.
CSCE 711	The cryptographic primitives and their correct use.	The part that works — misused: nonces, keys, error handling, composition.
CSCE 713 (this one)	Defects in code, and the techniques that find them.	Where the severe ones are, and where the tooling has theoretical limits rather than merely practical ones.

And all three courses converge on a single conclusion: **make the defect unexpressible rather than training people not to write it.** CSCE 701 Module 11 §3 reached it through defaults and the compliance budget; CSCE 711 Module 01 §4 reached it through misuse-resistant APIs and the historical record of cryptographic failures; this course reached it through Module 02 §4's safe languages and Module 10 §2's safe-by-construction interfaces. **Three independent routes to the same place is the semester's result,** and it is worth more than any individual technique in it.

4 A proportionate programme

- 1 · Sanitisers in continuous integration (Module 09) — **no source changes required, and it finds defects on the first day** in code that has passed its tests for years. **The cheapest real improvement available.**
- 2 · Fuzz the parsers (Module 08) — **where untrusted input meets manual memory handling,** with coverage reported rather than duration, and the corpus kept.
- 3 · Hardening flags (Module 12 §3) — **free, and verify them on the artefact rather than in the makefile.**
- 4 · Static analysis, on new code only (Module 07 §3's baseline), **with your own rules encoding your own invariants** (Module 07 §4), which is where the real yield is.
- 5 · **Then review the authorisation and parsing code** (Module 11 §1's two highest-yield categories), **and make the recurring defect unexpressible** (Module 10 §2). **Steps 1 to 3 are about a week's work and change the defect profile measurably** — and **step 5 is where the durable improvement is, which is why it is last rather than unimportant:** it requires knowing which defect recurs, and steps 1 to 4 are how you find out.

WHERE THIS COURSE LEAVES YOU

You can explain why vulnerabilities exist, name the classes that account for the severe ones, and recognise each of them in code you are reading — which is the practical skill, and it transfers across languages because the classes do.

You can run the techniques and read their output honestly: a fuzzing campaign measured by coverage rather than duration, static analysis triaged with written reasons, sanitisers used as the oracle rather than as a debugger, and review aimed specifically at what the tooling structurally cannot see.

And you know that the limits are theoretical rather than temporary — CSCE 627's undecidability results mean no tool will ever decide these properties, so the choice is always which way to be wrong, and a vendor promising otherwise is selling something.

The closing rule is the program's, unchanged across twenty-seven courses: state what you measured, state what you assumed, and never claim more than you established. In software security it means naming the technique, its coverage, and its blind spots — because 'the tools found nothing' states none of the three, and §2's table is what happens when it is accepted as though it did.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Klees et al. — Evaluating Fuzz Testing (free)	§1's fuzzing row, rigorously — what a campaign establishes and what is routinely claimed instead.
Bessey et al. — A Few Billion Lines of Code Later (free)	§1's static analysis row, from the field — honest about what the tools find and what customers believe they find.
Google — Building Secure and Reliable Systems, the concluding chapters (free PDF)	§3's convergence and §4's ordering , argued from operating experience.
The CWE Top 25, current edition (free)	§4's evidence — reread it annually and check your own programme's emphasis against it.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- For each of the five techniques**, write what it establishes and what it does not.

- 2 **Find a security claim made about software you use** and assess it against §2's table.
- 3 **Rewrite 'the scanner found nothing'** in the four-clause form.
- 4 **Take your own test coverage figure** and say what it does not measure.
- 5 **Write the honest claim** for your own fuzzing campaign.
- 6 **Write your limitation statement** — the classes your methods cannot see.
- 7 **Identify where the three courses converge**, in your own words.
- 8 **Compare your own programme** against §4's five steps and report the gaps.
- 9 **Revisit Module 01's last exercise** and compare.
- 10 **Project 2 is now due.** Submit the fuzzing campaign with coverage, the triaged static analysis, the manual review of the highest-risk component, the fixes offered upstream, and the limitation analysis.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 For five techniques, what does each establish and not establish?
- 2 What do all five have in common?
- 3 Why is 'the scanner found nothing' a statement about the scanner?
- 4 Give the four-clause honest form.
- 5 Give six overclaims and the correction to each.
- 6 Why is the coverage overclaim the most seductive?
- 7 Give three defensible claims.
- 8 What distinguishes an assessment from a tool run?
- 9 How does the semester divide, and where do the three courses converge?
- 10 Give the five-step programme and say where the durable improvement is.